

Join e sottoquery

Join avanzati

SELF JOIN e CROSS JOIN per casi meno comuni ma molto utili.

Quando le join normali non bastano

Non tutte le join servono a collegare due tabelle diverse. A volte devi unire una tabella con se stessa. Altre volte vuoi generare tutte le combinazioni possibili tra due insiemi.

Qui entrano in gioco `SELF JOIN` e `CROSS JOIN`.

SELF JOIN

Con `SELF JOIN` una tabella viene collegata a se stessa.

```
SELECT d.nome AS dipendente, r.nome AS responsabile
FROM dipendenti AS d
LEFT JOIN dipendenti AS r ON d.responsabile_id = r.id;
```

Questo succede perché nella tabella `dipendenti` ogni persona può avere un responsabile che è, a sua volta, un altro dipendente.

CROSS JOIN

`CROSS JOIN` crea tutte le combinazioni possibili tra due tabelle.

```
SELECT colori.nome, taglie.nome
FROM colori
CROSS JOIN taglie;
```

Se hai 3 colori e 4 taglie, il risultato avrà 12 righe.

Quando hanno senso

SELF JOIN è utile per strutture gerarchiche, come:

- dipendente e responsabile
- categoria e sotto-categoria
- persona e referente

CROSS JOIN è utile quando vuoi costruire tutte le combinazioni teoriche, per esempio colori e taglie di un catalogo.

CTE

Usare WITH per creare risultati temporanei leggibili e riutilizzabili nella query.

Dare un nome a un passaggio intermedio

CTE significa **Common Table Expression**. Il nome è tecnico, ma l'idea è semplice: crei un risultato temporaneo, gli dai un nome e lo usi subito dopo.

È come prendere appunti mentre risolvi un problema. Invece di tenere tutto in testa, scrivi un passaggio, lo chiami in modo chiaro e poi continui.

```
WITH ordini_grandi AS (  
    SELECT id, cliente_id, totale  
    FROM ordini  
    WHERE totale > 100  
)  
SELECT id, cliente_id, totale  
FROM ordini_grandi;
```

Qui **ordini_grandi** è un nome temporaneo. Dentro ci sono gli ordini con totale maggiore di 100.

La CTE esiste solo per questa query. Non crea una nuova tabella permanente nel database.

Perché rende le query più leggibili

Senza CTE, una query lunga può diventare difficile da seguire.

Con una CTE, invece, puoi separare il ragionamento:

1. prima preparo un gruppo di dati
2. poi uso quel gruppo nella query principale

Il vantaggio è soprattutto mentale. Leggi la query come una piccola storia.

Un esempio con una join

Troviamo prima gli ordini sopra 100 euro. Poi colleghiamoli ai clienti:

```
WITH ordini_grandi AS (  
    SELECT id, cliente_id, totale  
    FROM ordini  
    WHERE totale > 100  
)  
SELECT clienti.nome, ordini_grandi.totale  
FROM ordini_grandi  
INNER JOIN clienti ON clienti.id = ordini_grandi.cliente_id;
```

La CTE si occupa del primo passaggio: "quali sono gli ordini grandi?".

La query principale si occupa del secondo: "a quali clienti appartengono?".

Più CTE nella stessa query

Puoi anche creare più risultati temporanei:

```

WITH ordini_grandi AS (
    SELECT id, cliente_id, totale
    FROM ordini
    WHERE totale > 100
),
clienti_attivi AS (
    SELECT id, nome
    FROM clienti
    WHERE attivo = TRUE
)
SELECT clienti_attivi.nome, ordini_grandi.totale
FROM ordini_grandi
INNER JOIN clienti_attivi ON clienti_attivi.id = ordini_grandi.cliente_id;

```

Ogni CTE ha un nome e un compito. Questo aiuta molto quando una query cresce.

CTE o subquery?

Una subquery va benissimo quando il passaggio interno è piccolo.

Una CTE diventa più comoda quando vuoi dare un nome al passaggio, riusarlo o rendere la query più leggibile.

Non è una gara tra strumenti. Scegli quello che rende più chiaro il ragionamento.

Riepilogo rapido

- Una CTE nasce con `WITH`.
- Crea un risultato temporaneo con un nome.
- Vive solo per la durata della query.
- Aiuta a leggere query lunghe o divise in più passaggi.
- Spesso è più chiara di una subquery molto annidata.

Join

Come collegare tabelle diverse con `INNER JOIN`, `LEFT JOIN`, `RIGHT JOIN` e `FULL JOIN`.

Mettere insieme dati che vivono in tabelle diverse

In un database relazionale, le informazioni non stanno sempre tutte nella stessa tabella.

Per esempio, potresti avere una tabella `clienti` e una tabella `ordini`. La prima contiene le persone. La seconda contiene gli acquisti.

Da sole sono utili, ma insieme raccontano una storia più completa: **chi ha comprato cosa**.

Le `JOIN` servono proprio a questo: collegare tabelle diverse mentre leggi i dati.

Un primo esempio

Immagina queste due tabelle.

clienti

id	nome
1	Luca
2	Marta
3	Sara

ordini

id	cliente_id	totale
10	1	49.90
11	2	18.50

Nella tabella `ordini`, la colonna `cliente_id` dice a quale cliente appartiene ogni ordine.

Per mostrare il nome del cliente insieme al totale dell'ordine, scrivi:

```
SELECT clienti.nome, ordini.totale
FROM clienti
INNER JOIN ordini ON clienti.id = ordini.cliente_id;
```

Il risultato sarà:

nome	totale
Luca	49.90
Marta	18.50

Sara non appare perché non ha ordini collegati.

Cosa fa **INNER JOIN**

INNER JOIN tiene solo le righe che trovano una corrispondenza in entrambe le tabelle.

Nel nostro esempio:

- Luca ha un ordine, quindi appare.
- Marta ha un ordine, quindi appare.
- Sara non ha ordini, quindi non appare.

È come fare l'appello usando due liste e tenere solo i nomi che riesci ad abbinare.

La parte più importante: **ON**

Questa riga è il collegamento:

```
ON clienti.id = ordini.cliente_id
```

Significa:

"Abbinare un cliente a un ordine quando l'id del cliente è uguale al `cliente_id` salvato nell'ordine."

Se sbagli questa condizione, la join collega righe sbagliate. Per questo conviene leggerla sempre con calma.

Perché scriviamo **clienti.nome**

Quando usi più tabelle, possono esistere colonne con lo stesso nome.

Per esempio, sia `clienti` sia `ordini` hanno una colonna `id`. Scrivere `clienti.id` o `ordini.id` evita ambiguità.

È come usare nome e cognome quando in una stanza ci sono due persone chiamate Luca.

LEFT JOIN : tenere anche chi non ha corrispondenze

A volte vuoi vedere tutti i clienti, anche quelli che non hanno ancora fatto ordini.

In quel caso usi `LEFT JOIN` :

```
SELECT clienti.nome, ordini.totale
FROM clienti
LEFT JOIN ordini ON clienti.id = ordini.cliente_id;
```

Il risultato diventa:

nome	totale
Luca	49.90
Marta	18.50
Sara	NULL

`NULL` significa che manca un valore. Qui vuol dire: "per Sara non esiste un ordine collegato".

Quale join scegliere?

All'inizio concentrati su queste due:

- `INNER JOIN` quando vuoi solo le righe che si abbinano.
- `LEFT JOIN` quando vuoi tenere tutte le righe della tabella di sinistra.

Esistono anche `RIGHT JOIN` e `FULL JOIN` , ma puoi incontrarle più avanti. Nelle query quotidiane, `INNER JOIN` e `LEFT JOIN` coprono già tantissimi casi.

Riepilogo rapido

- Una `JOIN` legge insieme dati di tabelle diverse.

- `INNER JOIN` mostra solo le corrispondenze.
 - `LEFT JOIN` tiene anche le righe senza corrispondenza a destra.
 - `ON` spiega come collegare le tabelle.
 - Scrivere `tabella.colonna` evita confusione quando più tabelle hanno colonne simili.
-

Subquery

Come usare una query dentro un'altra per costruire richieste a più passaggi.

Una domanda dentro un'altra domanda

Una **subquery** è una query scritta dentro un'altra query.

Ti serve quando la risposta finale dipende da un passaggio intermedio. Prima trovi qualcosa. Poi usi quel risultato per fare la domanda principale.

Immagina di voler trovare i clienti che hanno fatto almeno un ordine sopra 100 euro. Puoi pensarla in due passi:

1. trova gli ordini sopra 100 euro
2. trova i clienti collegati a quegli ordini

In SQL diventa:

```
SELECT nome
FROM clienti
WHERE id IN (
    SELECT cliente_id
    FROM ordini
    WHERE totale > 100
);
```

La query interna è questa:

```
SELECT cliente_id
FROM ordini
WHERE totale > 100
```

Produce una lista di id cliente. La query esterna usa quella lista per trovare i nomi nella tabella `clienti`.

Perché usare una subquery

Una subquery è utile quando il problema si legge naturalmente a tappe.

Per esempio:

- trova i prodotti più costosi della media
- mostra i clienti che hanno fatto ordini
- cerca gli studenti iscritti a un certo corso

In tutti questi casi, una parte della query prepara il risultato per l'altra.

Subquery con una lista

Quando la query interna restituisce più valori, spesso usi `IN`.

```
SELECT nome
FROM clienti
WHERE id IN (
    SELECT cliente_id
    FROM ordini
);
```

Qui stai dicendo:

“Mostrami i clienti il cui id compare nella lista dei `cliente_id` presenti negli ordini.”

Subquery con un solo valore

A volte la query interna restituisce un valore solo.

Per esempio, questa query trova i prodotti che costano più della media:

```
SELECT nome, prezzo
FROM prodotti
WHERE prezzo > (
    SELECT AVG(prezzo)
    FROM prodotti
);
```

La query interna calcola la media dei prezzi. La query esterna mostra solo i prodotti con prezzo superiore a quella media.

Qui non usi **IN**, perché non stai confrontando con una lista. Stai confrontando con un solo numero.

Quando preferire una CTE

Le subquery sono comode, ma possono diventare difficili da leggere quando si annidano troppo.

Se senti che la query sta diventando una matrioska, spesso una CTE con **WITH** è più chiara. Le vediamo nella pagina dedicata.

Riepilogo rapido

- Una subquery è una query dentro un'altra query.
- Serve quando una domanda ha bisogno di un passaggio intermedio.
- Con **IN** confronti un valore con una lista.
- Con operatori come **>**, **<** o **=** confronti con un singolo valore.
- Se la query diventa troppo annidata, valuta una CTE.